МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №2

Тема: **«Ручное и автоматизированное тестирование»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения продемонстрировать на выбор применение технологии тестирования: ручное или автоматизированное.

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

Автоматизированное тестирование: сценарии выполняются программно (юнит-тесты, интеграционные тесты, end-to-end скрипты). Подходит для регрессий, повторяемых проверок и CI-пайплайнов.

Имеем класс GameLevelBuilder, отвечающий за размещение игровых сущностей на карте и сборку уровня:

```
class GameLevelBuilder:
    def __init__(self, factory: GameObjectFactory, size: tuple[int,
int]) -> None:
        self.factory = factory
        self.bounds = pygame.Rect(32, 82, size[0] - 64, size[1] - 114)
        self.player_start = self.bounds.center
        self._coins: list[tuple[int, int]] = []
        self._hazards: list[tuple[int, int]] = []

    def set_player_start(self, position: tuple[int, int]) ->
GameLevelBuilder:
        self.player_start = position
        return self

    def add_coin(self, position: tuple[int, int]) -> GameLevelBuilder:
        self._coins.append(position)
        return self

    def add_hazard(self, position: tuple[int, int]) -> GameLevelBuilder:
        self._hazards.append(position)
        return self

    def build(self) -> GameLevel:
        if not self.bounds.collidepoint(self.player_start):
            raise ValueError("Player start must be inside the level
bounds")
        if any(not self.bounds.collidepoint(point) for point in
self._coins + self._hazards):
            raise ValueError("All level objects must be inside the level
bounds")
        player = self.factory.create(ObjectKind.PLAYER,
self.player_start)
        coins = [self.factory.create(ObjectKind.COIN, point) for point
in self._coins]
```

```
        hazards = [self.factory.create(ObjectKind.HAZARD, point) for
point in self._hazards]
        if not isinstance(player, Player):
            raise TypeError("The object factory must create a Player for
ObjectKind.PLAYER")
        if any(not isinstance(coin, Coin) for coin in coins):
            raise TypeError("The object factory must create Coins for
ObjectKind.COIN")
        if any(not isinstance(hazard, Hazard) for hazard in hazards):
            raise TypeError("The object factory must create Hazards for
ObjectKind.HAZARD")
        objects = [player, *coins, *hazards]
        if any(not self.bounds.contains(obj.rect) for obj in objects):
            raise ValueError("All game object rectangles must fit inside
the level bounds")
        return GameLevel(self.bounds.copy(), player, coins, hazards)
```

Необходимо убедиться, что класс выдаст ошибку, если координаты какой-либо игровой сущности выходят за границы уровня. Поскольку вручную это проверить нельзя, нужно разработать юнит-тест:

```
def test_level_builder_rejects_out_of_bounds_objects(self):
    builder = GameLevelBuilder(ArcadeObjectFactory(), (960, 640))
    builder.add_coin((0, 0))
    builder.add_hazard((0, 0))
    with self.assertRaises(ValueError):
        builder.build()
```



Рисунок 1. Результат выполнения теста

Вывод:

Проделав работу, применил технологии ручного и автоматизированного тестирования для разработанного приложения.